

ML Start – домашнее задание #3

Тема 24: архитектура имеет значение – глубина vs ширина

Сорочан Дмитрий | [GitHub](#)

ПОСТАНОВКА ЗАДАЧИ

Цель работы – исследовать, как число скрытых слоёв и число нейронов влияют на аппроксимацию MLP. По теме 24 необходимо сравнить shallow и deep сети при одинаковом числе параметров и построить heatmap «ширина × глубина → качество». Используются все три типа данных из условия: **MNIST**, **Fashion-MNIST** и **синтетические функции**. Дополнительно MLP сравнивается с классическими моделями: Logistic Regression, Random Forest и LightGBM.

Исследуемая нейросеть намеренно простая:

$$x \rightarrow \text{Flatten} \rightarrow (\text{Linear} \rightarrow \text{ReLU})^d \rightarrow \text{Linear}(10).$$

BatchNorm, residual connections, scheduler и автоматический hyperparameter tuning не используются: иначе будет сложно отделить эффект архитектуры от эффекта дополнительных техник.

EDA И ДАННЫЕ

Оба image-датасета содержат 60 000 train и 10 000 test изображений размером 28×28 , 10 классов и grayscale-пиксели в диапазоне $[0, 255]$. В EDA проверяются баланс классов, случайные изображения, средние изображения классов, интенсивность пикселей, яркость, pixel variance, расстояния между средними классами и PCA-проекция.

MNIST визуально проще: большая часть изображения – фон, а классы задаются формой цифры. В Fashion-MNIST классы *T-shirt/top*, *Shirt*, *Pullover* и *Coat* сильнее похожи друг на друга, поэтому ожидается более низкое качество при одинаковом training protocol.

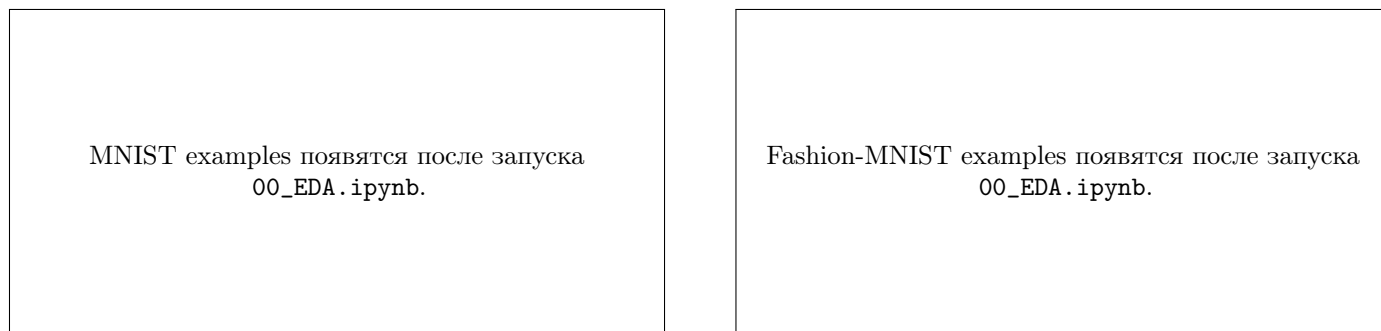


Рис. 1: Примеры объектов двух реальных датасетов.

ВАЛИДАЦИЯ И КОНТРОЛЬ LEAKAGE

Official test не используется при EDA и выборе архитектуры. Official train один раз делится на train/validation со стратификацией и фиксированным seed. Нормализация MLP рассчитывается только по train. Для большого architecture sweep используется одна и та же стратифицированная подвыборка 20 000 train-объектов и тот же validation.

- optimizer, learning rate и число эпох фиксированы внутри sweep;
- early stopping не применяется, чтобы training budget был одинаковым;
- width в parameter-matched эксперименте выбирается только по формуле числа параметров;
- test используется после architecture selection;
- классические baseline сравниваются на том же train size и validation split.

Основные метрики – **accuracy** и **macro-F1**. Дополнительно для классики считаются balanced accuracy и время обучения.

ГЛУБИНА, ШИРИНА И ЧИСЛО ПАРАМЕТРОВ

Для d скрытых слоёв одинаковой ширины w число обучаемых параметров равно

$$P(d, w) = (784 + 1)w + (d - 1)(w + 1)w + (w + 1) \cdot 10.$$

Поэтому сеть с большей глубиной при фиксированном w одновременно имеет больше параметров. Обычная heatmap отвечает на вопрос «какая конкретная конфигурация работает лучше», но не доказывает, что выигрыш вызван именно глубиной.

В основном sweep проверяются

$$d \in \{1, 2, 3, 5, 10, 20, 50, 100\}, \quad w \in \{8, 16, 32, 64, 128, 256\}.$$

MNIST heatmap будет создана в
01_architecture_research.ipynb.

Fashion-MNIST heatmap будет создана в
01_architecture_research.ipynb.

Рис. 2: Heatmap «width × depth → validation accuracy».

Кроме heatmap строятся accuracy vs depth, accuracy vs width, accuracy vs parameter count и отдельные learning curves train/validation loss и accuracy. Это позволяет отличить недостаточную выразительность от проблем оптимизации.

SHALLOW VS DEEP ПРИ ОДИНАКОВОМ БЮДЖЕТЕ

Для более чистого сравнения задаются бюджеты примерно 50k, 150k и 500k параметров. Для каждого $d \in \{1, 2, 5, 10, 20, 50, 100\}$ выбирается целая ширина w , дающая ближайшее число параметров. Validation score не участвует в выборе width.

Parameter-matched MNIST result появится после
запуска эксперимента.

Parameter-matched Fashion-MNIST result появится
после запуска эксперимента.

Рис. 3: Validation accuracy при сопоставимом числе параметров.

Если deep-сеть выигрывает в обычной heatmap, но не выигрывает при одинаковом parameter budget, нельзя приписывать преимущество глубине: причиной мог быть просто больший размер модели.

УНИВЕРСАЛЬНАЯ АППРОКСИМАЦИЯ

Теорема об универсальной аппроксимации в классической форме утверждает: feed-forward сеть с одним скрытым слоем и подходящей нелинейной активацией при достаточной ширине может сколь угодно точно приблизить непрерывную функцию на компактном множестве.

Это **теорема существования**, а не инструкция по обучению. Она не говорит:

- сколько нейронов потребуется shallow-сети;
- будет ли такое представление параметрически эффективным;
- найдёт ли optimizer нужные веса;
- насколько быстро сеть обучится;
- как она будет обобщать на новых данных.

Для функций с композиционной структурой глубина может дать существенно более компактное представление. Одновременно plain MLP глубиной 50–100 слоёв без residual connections может быть труднее оптимизировать. Поэтому формулировки «глубже всегда лучше» и «одного слоя всегда достаточно» обе некорректны как практические выводы.

КЛАССИЧЕСКИЕ МОДЕЛИ

В отдельном ноутбуке сравниваются Logistic Regression, Random Forest и LightGBM. Гиперпараметры намеренно фиксированы и не подбираются по validation: классика нужна как понятный baseline, а не как второй большой tuning-проект.

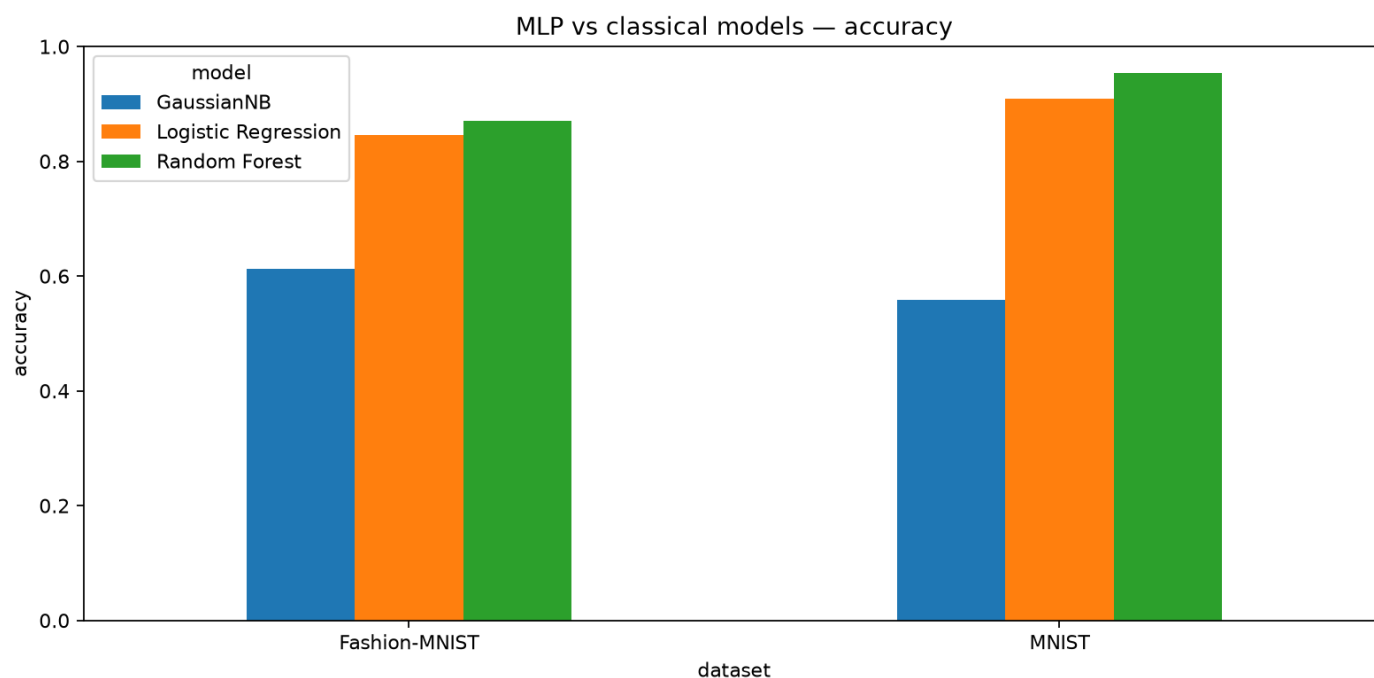


Рис. 4: Validation accuracy MLP и классических моделей.

Сравнение проводится также по macro-F1 и времени обучения. Для лучшей классической модели и финальной MLP строятся confusion matrices.

СИНТЕТИЧЕСКИЕ ФУНКЦИИ

Используются три функции:

$$f_1(x) = \sin(3x) + 0.3 \cos(7x),$$

$$f_2(x) = \sin(2x + 0.8 \sin(5x)),$$

$$f_3(x) = 0.6|x| + 0.4 \sin(6x).$$

На f_1 строится отдельная heatmap depth $\times$ width по MSE; на всех трёх сравниваются shallow/deep аппроксимации и learning curves.

Synthetic heatmap будет создана в
03_synthetic_functions.ipynb.

График аппроксимации композиционной функции
появится после запуска.

Рис. 5: Синтетический эксперимент: ошибка и форма аппроксимации.

РЕЗУЛЬТАТЫ И ИНТЕРПРЕТАЦИЯ

После выполнения ноутбуков численные таблицы сохраняются в `report/*.csv`, а графики – в `report/figures/`. В отчёте принципиально не зафиксированы заранее придуманные числа: итоговые значения должны соответствовать реальному запуску на конкретном окружении.

Численная таблица автоматически появится здесь после последовательного запуска всех ноутбуков и повторной компиляции отчёта.

Корректный вывод строится в таком порядке:

1. найти лучшую пару depth/width по validation;
2. проверить, сохраняется ли преимущество в parameter-matched сравнении;
3. посмотреть learning curves и понять, является ли провал deep-сети проблемой оптимизации;
4. сравнить лучшую MLP с RF/LightGBM/Logistic Regression;
5. использовать official test после architecture selection;

6. проверить, повторяется ли качественная картина на Fashion-MNIST и синтетических функциях.

Главная идея работы: **архитектуру нельзя оценивать только по одному числу accuracy**. Нужно разделять выразительность, число параметров и оптимизируемость сети.

ВОСПРОИЗВОДИМОСТЬ

Код целиком находится в четырёх Jupyter Notebook без отдельного 'src/' и тестовой обвязки. 'SEED=143' фиксирован, данные автоматически скачиваются 'torchvision' в пользовательский cache, результаты сохраняются рядом с LaTeX-отчётом. Инструкция запуска на Linux приведена в 'README.md'.